

USER_INTENT SPEC: Distributed ThreadMail System

Part 1 — Main Purpose, Setup Constraints, and Design Reasoning

1.1 Main Purpose

The system is a low-cost, cloud-folder-based coordination system for humans and AI agents.

Its purpose is to let multiple humans and AI agents coordinate work through durable, topic-oriented message threads without requiring:

- public IP addresses
- DDNS
- VPN
- hosted server
- hosted database
- cloud provider APIs
- realtime delivery
- high-frequency polling
- chat-style instant messaging

The system should behave more like a delayed mail-sorting system than a chat application.

Each participant publishes messages into its own controlled outbox. A single global coordinator later distributes those messages into thread folders. Once messages are distributed into a thread, other participants can read and respond.

The system is intended for low-volume coordination, not high-throughput messaging.

1.2 Core Communication Model

The system uses a cloud-synced filesystem folder as the shared transport layer.

The cloud provider may be:

- Google Drive
- OneDrive
- iCloud Drive
- Dropbox
- Syncthing
- local shared folder
- another filesystem-like sync provider

The system must not depend on provider-specific APIs.

All interaction with shared storage should happen through ordinary filesystem operations:

- list folder
- read file
- write new file
- copy folder
- move folder
- rename staging folder into final folder
- optionally watch local filesystem changes

The cloud folder is not a database. It is a durable mailbox and filing cabinet.

1.3 Main Architectural Concept

The system has four major logical areas:

```
local drafts
  ↓
per-USER cloud outbox
  ↓
global coordinator distribution
  ↓
thread folders
  ↓
archive
```

The intended lifecycle is:

```
DRAFT
  local-only, editable, private

PUBLISHED_TO_OUTBOX
  immutable package written by local service

DISTRIBUTED_TO_THREAD
  coordinator copied package into target thread

ACKED
  coordinator wrote a receipt for the source USER

ARCHIVED
  thread closed and packaged for long-term storage
```

1.4 USER Concept

A USER may be:

- a human
- an AI agent
- a local automated process

A USER is not necessarily the same thing as a machine.

Multiple USERS may exist on the same machine.

Each USER has:

- its own local draft area
- its own cloud-visible outbox
- its own local message counter
- its own receipts folder

Each USER is responsible for assigning its own local message IDs. Since each USER writes only into its own outbox, no distributed local-message-ID conflict exists.

1.5 Outbox Concept

Each USER has one private cloud-visible outbox.

The outbox is an application-owned queue. Humans and AI agents must not manually edit the outbox folder.

Humans and agents interact with the outbox only through:

- WebApp UI
- local API
- Python client library
- JavaScript client library
- approved local service

The local service must refuse to overwrite existing outbox packages.

Outbox packages are immutable after publishing.

If a human or agent wants to revise a message, it must publish a new message package.

1.6 Thread Concept

The system is topic/thread-oriented, not private-message-oriented.

A thread is a temporary work context.

A thread:

- has a unique thread ID
- has a topic/title
- has a README-like description
- remains open for days or weeks
- receives distributed messages
- may contain attachments and multimodal content
- is marked DONE when mature
- is not reopened after DONE
- may be extended by creating a new thread linked to the old thread

The system should not model chat rooms or direct messages as the primary abstraction.

1.7 DONE and Follow-Up Behavior

Threads do not reopen.

If additional work is needed after a thread is DONE, the system creates a new thread that references the old one.

Example:

```
T_20260627_A1B2 = original thread  
T_20260705_C9D3 = follow-up thread, extends T_20260627_A1B2
```

This avoids lifecycle ambiguity and prevents closed records from being mutated.

1.8 Multimodal Messages

Messages may include:

- Markdown text
- images
- PDFs
- audio
- video
- CSV files
- JSON files
- arbitrary attachments

Large binary content should not be embedded directly in JSON metadata.

A message package should contain:

```
message.json  
body.md  
attachments/
```

The metadata should include MIME/media type information so the WebApp can render or play attachments with suitable controls.

1.9 Global Coordinator Concept

The global coordinator is a background distributor.

In version 1, there is only one coordinator.

The coordinator:

- scans registered USER outboxes
- ignores unregistered outboxes
- distributes immutable outbox packages into target thread folders
- assigns distributed timestamp and distributed counter
- writes receipts
- updates indexes periodically
- archives DONE threads
- maintains a local ledger/cache
- can rebuild state from the filesystem

The coordinator is not required for USERS to publish to their outboxes.

If the coordinator is down:

- USERS can still draft locally
- USERS can still publish immutable outbox messages
- outbox messages wait
- no new messages appear in thread folders
- agents naturally stop when no new distributed work is visible
- when the coordinator returns, it catches up

This is an intentional design.

1.10 Message Ordering

There are two different orders:

```
USER-local order  
  assigned by each USER in its own outbox
```

thread distribution order
assigned by the coordinator when distributing into a thread

The official thread-visible order is the coordinator distribution order.

Client device clocks may not be synchronized. Therefore client timestamps must not be used as the primary global ordering source.

The distributed timestamp should mean:

the time the coordinator first observed and accepted a stable outbox package

This is a practical provider-neutral substitute for “time the file hit the cloud drive.”

The final thread message filename may include:

- distributed timestamp
- coordinator counter
- source USER ID
- source local message ID

Example:

20260627T161501Z_000001_agent_reviewer_U000017/

The counter resolves same-timestamp collisions.

1.11 No-Loss Requirement

No message should be lost.

The system must assume:

- cloud sync may delay files
- cloud sync may duplicate events
- files may appear slowly
- filesystem watchers may miss events
- coordinator may crash
- coordinator may restart
- network may go down
- USERS may publish while coordinator is offline

Therefore:

- outbox packages are immutable
- published outbox packages are not deleted by USERS
- coordinator processes idempotently
- source identity is stable
- receipts are written after distribution
- processed messages can be retained or archived
- recovery can rebuild from outboxes and thread folders
- periodic scans are the correctness path
- filesystem watchers are only a fast path

The stable source identity is:

```
source_user_id + source_local_message_id
```

This identity must remain attached to every distributed thread message.

1.12 Cloud-Friendliness Requirement

The system should avoid abusing cloud storage.

It should avoid:

- frequent heartbeats
- tight polling loops
- lock-file churn
- repeated overwrites
- editing the same global index constantly
- using cloud sync as a realtime database

Preferred behavior:

- append-only outbox packages
- batched coordinator scans
- batched distribution
- periodic index refresh
- low-frequency cleanup
- archival of closed threads
- no provider-specific API calls

Cloud storage should be treated as postal mailboxes and filing cabinets, not as a database.

1.13 Version-1 Scope

Version 1 should use:

- one global coordinator
- manual registered-outbox config
- local WebApp
- local client service
- Python client library
- optional JavaScript client library
- filesystem-based storage
- local drafts
- immutable outbox packages
- thread folders
- receipts
- archive support

Version 1 should not include:

- redundant coordinators
- leader election
- shared-secret coordinator negotiation
- provider-specific Drive/OneDrive/iCloud APIs
- realtime messaging
- public server
- hosted database
- mobile app

1.14 Version-2 Considerations

Version 2 may add:

- mobile app
- coordinator packaged as Android foreground app
- redundant coordinators
- slow heartbeat
- advisory lease
- leader election
- shared-secret coordinator identification
- coordinator cluster folder
- epoch/fencing support

These must be isolated so v1 is not overcomplicated.

Part 2 — WebApp User Intent

2.1 WebApp Purpose

The WebApp is the human-facing local interface.

It allows humans to:

- read distributed threads
- create and edit local drafts
- attach files
- publish immutable messages to their own USER outbox
- view pending outbox messages
- view receipts
- request new threads
- mark or request DONE
- browse archived threads
- inspect attachments and multimodal content

The WebApp is not the global coordinator.

The WebApp is a standalone local application. It may be implemented as:

- Python local app serving a browser UI
- standalone JavaScript/Electron/Tauri app
- another local GUI

For v1, a Python local WebApp is preferred.

2.2 WebApp Operating Model

The WebApp runs locally and is configured with:

```
user_id
display_name
local draft root
cloud outbox path
receipt path
thread root
index path
archive path
```

The WebApp owns the USER's local draft workflow and outbox publishing workflow.

The WebApp must not require cloud provider APIs.

The WebApp must not assume the global coordinator is always online.

2.3 Human Draft Behavior

Human users should never manually write into their outbox.

The WebApp provides a local draft area where users can repeatedly save, revise, attach, and preview without confusing the system.

Drafts are:

- local
- private to the local app
- mutable
- not visible to other participants
- not processed by the coordinator

A draft becomes visible to the system only when the user presses Publish.

2.4 Publish Behavior

When the user publishes a draft, the WebApp creates a new immutable outbox package.

The WebApp must:

- generate a new USER-local message ID
- create a staging package outside the final outbox location
- copy `message.json`, `body.md`, and attachments into staging
- validate that the final outbox folder does not already exist
- move or rename staging into the outbox as the final package
- refuse overwrite
- mark the draft as submitted
- keep the draft or submitted snapshot for local recovery

After publishing, the outbox package must be read-only from the WebApp's perspective.

If the user edits the original draft later, publishing again creates a new outbox package with a new local message ID.

2.5 WebApp Read Behavior

The WebApp reads messages only from distributed thread folders.

A message in a USER outbox is not considered readable by other USERS until the coordinator distributes it into the target thread.

The WebApp may show the local USER's pending outbox messages separately, but it must label them as pending/not distributed.

2.6 WebApp Thread View

The WebApp should provide:

```
Active Threads list
Thread detail view
Message timeline
Message composer
Attachment panel
Pending outbox panel
Receipts/status panel
Archive browser
```

The active thread list can rely on shallow metadata:

- thread ID
- title
- topic
- description
- status
- updated time
- participants/USERS if available

Active search does not need full-text indexing in v1.

Search over title/topic/description is sufficient.

2.7 WebApp Multimodal Rendering

The WebApp must display or play message attachments according to media type.

Recommended rendering rules:

```
image/*
  show image preview

audio/*
  show audio player

video/*
  show video player
```

```
application/pdf
  show open/download button or embedded PDF viewer

text/plain
  show text preview

text/markdown
  render Markdown preview

text/csv
  show table preview if practical, otherwise download/open button

application/json
  show structured preview if practical

unknown binary
  show attachment button
```

The UI should not assume attachments are safe or small. It should provide conservative open/download controls when needed.

2.8 WebApp Coordinator Actions

The WebApp may include a coordinator control screen if running on the coordinator machine, but this should be separate from normal user messaging.

Coordinator-facing WebApp actions may include:

- start scan now
- show last scan time
- show registered outboxes
- show pending messages
- show distributed count
- show dead-letter messages
- show errors
- trigger archive job
- rebuild index
- rebuild ledger from filesystem

The WebApp must make clear when the user is acting as a normal USER versus managing the coordinator.

2.9 WebApp Non-Goals for v1

The WebApp should not implement:

- real-time chat
- direct manual outbox editing

- direct cloud provider API integration
 - user registration automation
 - complex permission system
 - redundant coordinator control
 - mobile UI
-

Part 3 — Global Coordinator User Intent

3.1 Coordinator Purpose

The global coordinator is the only component that distributes USER outbox messages into readable thread folders.

It acts as a slow durable mail sorter.

It does not create user drafts. It does not provide normal human messaging UI. It does not need to be realtime.

3.2 Coordinator Input

The coordinator reads:

```
config/registered_outboxes.txt
users/<user_id>/outbox/
threads/
index/
archive/
```

The registered outboxes file is human-managed offline.

For v1, this can be a simple text file containing one registered outbox path per line.

Example:

```
users/human_wayne/outbox
users/agent_reviewer/outbox
users/agent_builder/outbox
```

The coordinator scans only listed outboxes.

It must ignore all unregistered outboxes and stray folders.

3.3 Coordinator Output

The coordinator writes:

```
threads/<thread_id>/messages/  
users/<user_id>/receipts/  
index/  
archive/  
coordinator/ledger or coordinator state  
coordinator/dead_letter/
```

The coordinator is the only component allowed to:

- distribute messages into thread folders
- write receipts
- update active indexes
- move packages to processed storage
- archive threads
- delete or clean up system-managed files according to policy

3.4 Coordinator Distribution Behavior

The coordinator periodically scans registered outboxes for immutable message packages.

For each outbox package, it should:

1. detect package
2. verify package is stable
3. validate package structure
4. read source USER ID and source local message ID
5. check whether that source identity was already distributed
6. validate target thread exists and is open
7. assign distributed timestamp
8. assign distributed counter
9. copy package into target thread message folder
10. write receipt
11. update ledger/cache
12. optionally move or mark package as processed later

The coordinator must be idempotent.

If it sees the same source message twice, it must not distribute it twice.

If receipt is missing for an already distributed message, it may rewrite the missing receipt.

3.5 Coordinator Timestamp and Counter

The official distributed timestamp is the coordinator-observed time when the coordinator accepts the stable outbox package.

The official thread order is by coordinator distribution order.

The coordinator assigns a counter for each distributed message, preferably per thread.

Final distributed message folder name may include:

```
distributed_timestamp  
distributed_counter  
source_user_id  
source_local_message_id
```

Example:

```
20260627T161501Z_000001_agent_reviewer_U000017/
```

3.6 Coordinator Ledger

The coordinator may maintain a local SQLite ledger for efficiency.

However, the ledger must not be the only truth.

The coordinator must be able to rebuild its state from the filesystem by scanning:

- distributed thread messages
- outbox packages
- receipts
- archive metadata

The core deduplication key is:

```
source_user_id + source_local_message_id
```

Recommended ledger fields:

```
source_user_id  
source_local_message_id  
target_thread_id  
distributed_filename
```

```
distributed_counter
distributed_at
status
receipt_path
content_hash
```

3.7 No-Loss Coordinator Rules

The coordinator must never delete an outbox package before safely distributing it and writing a receipt.

Safe order:

```
read outbox package
copy to thread
verify copy
write receipt
record ledger
then optionally mark processed or move later
```

Cleanup should be delayed and conservative.

If the coordinator crashes mid-process, a later scan should repair the state.

3.8 Dead-Letter Behavior

Invalid packages should not block the whole coordinator.

Malformed or rejected packages should be moved or copied into dead-letter storage with an error explanation, or left in place with a rejected receipt.

Possible dead-letter reasons:

- malformed JSON
- missing body file
- missing attachment
- unknown target thread
- closed thread
- duplicate source ID conflict
- invalid media metadata
- unreadable package

The coordinator should continue processing other messages.

3.9 Index Behavior

The coordinator may maintain:

```
index/active_threads.json  
index/archive_index.json
```

Indexes are convenience files, not the source of truth.

They do not need to be updated after every message. They may be refreshed periodically or after a batch.

The source of truth is:

```
thread folders  
distributed messages  
outbox packages  
receipts  
archives
```

3.10 Archive Behavior

When a thread is DONE and mature, the coordinator may archive it.

Archiving should create a durable package, such as:

```
archive/T_<thread_id>.zip
```

The archive should contain:

```
README.md  
manifest.json  
messages/  
attachments/  
final_summary.md if available  
hash_manifest.json if available
```

After archive creation and optional grace period, the live thread folder may be moved or removed according to policy.

3.11 Coordinator Timing

The system should avoid cloud-storage abuse.

Suggested v1 timing:

```
normal scan interval: 15-30 minutes
index refresh: periodic or batch-based
deep recovery scan: 4-12 hours
archive cleanup: daily or weekly
```

Filesystem watcher may be used as a fast path, but periodic scan is the correctness path.

3.12 Coordinator Version-1 Non-Goals

Version 1 coordinator should not implement:

- redundant coordinator election
- heartbeat protocol
- shared-secret negotiation
- leader lease
- epoch fencing
- cloud provider APIs
- realtime delivery
- high-frequency locking
- per-message distributed claims

These belong to version 2.

Part 4 — Client Library User Intent

4.1 Client Library Purpose

The client library provides a common programmatic interface for humans, AI agents, scripts, and the WebApp.

It should hide filesystem details and enforce protocol rules.

The client library should exist in at least one primary implementation:

- Python module for v1

A JavaScript/TypeScript library may be added for WebApp/mobile/client compatibility.

The WebApp should use the same client library or the same core protocol layer.

4.2 Client Library Responsibilities

The client library should support:

```
draft creation
draft editing
attachment management
message package creation
local USER message ID assignment
safe outbox publish
thread reading
receipt reading
pending status
archive reading
basic validation
```

The client library must enforce:

- no overwrite
- immutable outbox packages
- stable local message IDs
- package structure consistency
- attachment metadata correctness
- separation between draft and published package

4.3 Client Configuration

A client instance should be configured with:

```
user_id
display_name optional
local_draft_root
outbox_root
receipt_root
thread_root
index_root
archive_root
```

Example conceptual config:

```
{
  "user_id": "agent_reviewer",
  "display_name": "Reviewer Agent",
  "local_draft_root": "C:/ThreadMail/drafts/agent_reviewer",
  "outbox_root": "G:/CloudSync/threadmail/users/agent_reviewer/outbox",
  "receipt_root": "G:/CloudSync/threadmail/users/agent_reviewer/receipts",
  "thread_root": "G:/CloudSync/threadmail/threads",
  "index_root": "G:/CloudSync/threadmail/index",
```

```
"archive_root": "G:/CloudSync/threadmail/archive"
}
```

4.4 Draft API Intent

The client library should allow:

```
create_draft(thread_id, message_type)
update_draft_body(draft_id, body)
add_attachment(draft_id, path, media_type optional)
remove_attachment_from_draft(draft_id, attachment_id)
list_drafts()
read_draft(draft_id)
delete_local_draft(draft_id)
```

Drafts are local and mutable.

Draft changes must not affect any published outbox package.

4.5 Publish API Intent

The client library should allow:

```
publish_draft(draft_id)
```

Publishing should:

1. assign next USER-local message ID
2. build immutable package in staging
3. include message metadata
4. include body file
5. copy attachments
6. validate package
7. refuse overwrite
8. atomically finalize package into outbox if practical
9. mark local draft as submitted
10. return source identity

The returned source identity is:

```
source_user_id
source_local_message_id
outbox_package_path
```

4.6 Message Package Format

The client library should create packages like:

```
users/<user_id>/outbox/  
  U000017_<unique_suffix>/  
    message.json  
    body.md  
    attachments/  
      A000001_image.png  
      A000002_audio.m4a
```

`message.json` should include:

```
schema_version  
source_user_id  
source_local_message_id  
target_thread_id  
message_type  
body_file  
attachments  
local_created_at informational  
content_hash optional
```

Attachment metadata should include:

```
attachment_id  
original_filename  
stored_filename  
media_type  
relative_path  
display_hint optional
```

4.7 Read API Intent

The client library should allow:

```
list_active_threads()  
read_thread(thread_id)  
list_thread_messages(thread_id)  
read_message(thread_id, message_id or folder)  
list_attachments(message)  
open_attachment_reference()
```

```
list_receipts()
list_pending_outbox()
```

The client reads distributed messages from thread folders.

The client may also read its own outbox packages to show pending state.

4.8 Receipt API Intent

The client library should detect coordinator receipts.

Receipt status may include:

```
distributed
rejected
dead_letter
duplicate
archived
```

A distributed receipt should tell the client:

```
source_user_id
source_local_message_id
target_thread_id
distributed_filename
distributed_at
distributed_counter
```

The WebApp can use this to show:

```
Pending
Distributed as #N
Rejected: reason
```

4.9 Thread Creation and DONE Requests

The client library may support request-style messages for:

```
create_thread
mark_done
attach_followup_thread
```

In v1, these can be represented as normal outbox packages with special `message_type` or `request_type`.

The coordinator decides how to apply them.

4.10 Storage Abstraction

The client library should not know whether storage is Google Drive, OneDrive, iCloud, Dropbox, Syncthing, or local disk.

It should depend on a storage abstraction that supports:

```
list
read
write_new
copy_tree
rename_or_finalize
exists
stat
```

No cloud API is required.

4.11 Client Library Non-Goals for v1

The client library should not implement:

- global coordinator behavior
- leader election
- cloud provider API integration
- user registration workflow
- realtime push messaging
- distributed locks
- direct mutation of thread folders
- manual overwrite of outbox packages

4.12 Key Client Invariant

The client library owns this invariant:

```
Drafts are mutable.
Outbox packages are immutable.
Published packages are never overwritten.
Distributed thread messages are coordinator-owned.
```

This invariant must be enforced for humans, agents, and WebApp users equally.